

RENDERING: pre-rendering/offline VS realtime

COMPUTER ANIMATION (descriz azioni, es. luce Pixas) vs COMPUTER SIMULATION (leggi fisiche)

PROCESSO GRAFICO INTERATTIVO (move camera &/|| 3D models + render + display) vs ANIMATION (+ imgs)

GRAFICA → IMG come PRODOTTI FINALI (3DWeb, entertainment) o come AUSILIO (CAD)

HARDWARE:

- DISPLAY (CRT, tubo catodico) con fosfori e FrameBuffer → oggi FB Full Color, 24 piani colore
- GPU (ex DPU): processori specializzati per INTENSE DATA // COMPUTATIONS, hanno poca cache
- INPUT (tastiera, mouse, touchscreen)

INTERATTIVITÀ → POLLING + CODA EVENTI + EVENT DRIVEN

RASTER GRAPHIC/BITMAP: img come griglie pixel. ↑risoluz ↓diff tra img e rappresentazione

≠ primitiva punto

ALG. LINEA INCREMENTALE: disegno coord intere su viewport/schermo usando disegno continuo a pixel 4/8-connn (trovo pto partendo dal prec). Si vedono dentellature: serve antialiasing

ALG. LINEA BRESENHAM: disegno coord floating point su window, poi trasformo in viewport → **nelle GPU**

TRASFORMAZIONI GEOMETRICHE: xyzO sdr cartesiano DXorso, def. Spazio vettoriale RN in cui definire GEOMETRIA e TOPOLOGIA e fare trasformazioni come rotazione, traslazione, scala.

SPAZIO AFFINE: estensione spazio vettoriale con concetto di **PUNTO**. Ho una nuova coordinata, si usano coordinate omogenee (1 pto, 0 vettore). TRASF AFFINI: matrici r, t, s, inverse + trasformazioni composte: ORDINE TRS

TRASFORMAZIONI DI VISTA: SDR Oe SXorso 3D, SDR mondo DXorso 3D (coord SFERICHE $[\theta, \phi, D]$), SDR Window 2D

Punti 2D si possono rappr nel SDR affine con $[\alpha, \beta, \gamma]$ **COORD BARICENTRICHE:** se sono ≥ 0 contemp, interne al triangolo

RASTERIZZAZIONE: coloro pti interni usando coord baric. **INTERPOLAZIONE COLORI:** colore $\forall$ vertice, combino con coord baric

ALG. RASTERIZZAZIONE: det. seq orizzontali pixel del poligono con ALG LINEA INCREMENTALE + Edge Table + Active ET.

PROIEZIONE PARALLELA vs PROSPETTICA (1,2,3 pti di fuga) → Per det Window, simulo ampiezza cono di vista Oe (**FRUSTUM**)

WIREFRAME: ambiguo → **ALG. ELIMINAZIONE PARTI NASCOSTE:**

- BACK FACE CULLING (facce convesse triang nascoste non vengono disegnate)
- PITTORE (disegno una geometria sopra all'altra)
- VISIBILITY BUFFER (= pittore ma con regione di invisibilità)
- ZBUFFER (FBuffer per colori, Zbuffer per profondità. Estendo scanline, **nelle GPU**)

ALG. ZBUFFER/DEPTH BUFFER: uso segm di estremi in f. parametrica x texture/colori/proiez //, mentre x prospettiva serve trasformazione **DA FRUSTUM A CUBO** (così posso usare regole proiez //):

- TEOREMA X RETTE: scelgo A e B in modo che la piramide sia mappata su cubo
- TEOREMA PER PIANI: uso solo triangoli, ma stesso meccanismo

PIPELINE VISTA: SDR obj → SDR Oe, Scala frustum, Clipping Front&Back, SDR Oe → SDRpiano proiezione, Clipping lati, Win → View

CLIPPING PTI: controllo range, **CLIPPING LINEE:** **ALG. COHEN SUTHERLAND** 2D (9 regioni, 4bit codes) / 3D (27 regioni, 6bit codes)

Poligoni clippati non rimangono gli stessi → **ALG. SUTHERLAND-HODGMAN 2D/3D:** taglio in base al lato della Window, prima su frustum back/front, poi su cubo lati, e triangolarizzo dopo ogni taglio

TEXTURE: img x colorare superficie, TEXTURE MAPPING: applico img a superficie, PARAMETRIZZAZIONE (coord text → coord obj).

ZBUFFER implica trovare texel $\forall$ pixel con **INVERSE MAPPING** ($t > p$ = minification, $t < p$ = magnification)

TECNICHE ELABORAZIONE IMG:

- **NN:** $\forall$ pixel, trovo texel+vicino → **PIXELATION**
- **BILINEAR INTERPOLATION:** $\forall$ pixel, coloro 4 texel + vicini → **BLUR**
- **BICUBIC INTERPOLATION:** 4x4pixel+vicini (std in PS)

MIP MAPPING (se $2^n \times 2^n$ texel: collezione texture), **SOLID TEXTURE** (immersione), **S-MAPPING** (uso superficie S),

ENVIRONMENT MAPPING (img da raggio riflesso), **BUMP MAP** (deformo normali), **MULTITEXTURING**, **SMOKE SIMULATION**

OPENGL → pipeline fissa fino 1.5, programmabile con shaders da 2.0 + OpenGLSL

WEBGL2

- Implementa OPENGL 3 + Javascript + OpenESSL → supp mobile, no compilazione, gestione mem auto, integr in HTML5
- Coord $[-1, -1, -1]$ $[1, 1, 1]$, usa drawArrays/drawElements, solo PUNTI, LINEE, TRIANGOLI. Buffer Objects: VBO e IBO
- VERTEX SHADER $\forall$ vertice, FRAGMENT SHADER $\forall$ fragment(triangolo) x colori/texture/luci (ATTRIB, UNIFORMS, VARYING)
- **TRASF VISTA:** SDR obj → SDR Oe con **-ze**. Matrix M parametri vista, Matrix P param prospettiva

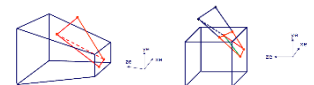
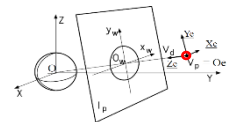
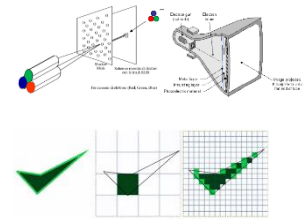
ILLUMINAZIONE: luce assorbita/riflessa/rifratta. **RENDERING EQUATION:** colore e qtà luce danno colore che umano vede.

Modelli locali (luce su obj)/globali (luce fra obj) → **MODELLO ILLUMINAZIONE PHONG:** locale, 3 componenti

- **KA:** ambiente, effetto luce che arriva
- **KD:** riflessione diffusa, tipica superfici OPACHE, cambia se cambia angolazione luce, Oe sta dove vuole
- **KS:** riflessione speculare, tipica superfici LUCIDE, cambia se cambia angolazione Oe

ALG. SHADING: **FLAT** (colore $\forall$ faccia, spigoli evidenti, Mach band), **GOURAUD** (applico luce + interpolaz colori vertici), **PHONG** (interpolaz normale vertici usando SLERP per normali equispaziate + applico luce). G e P eliminano spigoli vivi → doppi vertici

SHADERS: colore pixel/fragment dipende da POSIZ/PROPR **LUCE** (direzionale, puntiforme, spotlight), MATERIALE, MODELLO LUCE



ALG. SHADOWING: **OMBRE SU PIANI** (proj obj su piano, no per auto-ombre), **SHADOW BUFFER** (render pov luce con depth buffer + render pov Oe. Produce Aliasing: scalettature). **HARD SHADOWS** → **SOFT SHADOW** con + sorgenti, blur ombre, tecniche apposta

MESH 3D POLIGONALI (BREP) = obj approssimati da contorno **2-MANIFOLD** (intorno pti è omeomorfo ad un disco piano, posso fare deformazioni senza strappi). **GENERE SUPERFICIE:** n curve semplici chiuse disgiunte senza sconnettere. Se orientabile, n buchi

OPERATORI EULERO: trasformo vertici mesh rispettando topologia.

PER FRAGMENT OPERATIONS: tra Raster e Frame Buffer nell'architettura WebGL. **SCISSOR:** restringo disegno viewport, **MULTISAMPLE:** render scena risoluz ↑ in alcune zone, **STENCIL:** x riflessione, es specchio d'acqua, **DEPTH:** Zbuffer, **BLENDING:** trasparenze (modifico α), **DITHERING.**

CURVE BEZIER: f. param def da CP e da basi, def in GUSCIO CONVESSO dei CP, invarianti a trasf affini (tsr, deformazione shear)

FRENET FRAME: SDR ortogonale $\forall$ pto curva con Tangente unitaria, Binormale, Normale unitaria.

CURVATURA: grado scostamento curva da andamento rettilineo → COMB per progettare forme

RAPPRESENTARE UNA CURVA: specificare CP, definire parametrizzazione, definire mapping polinomiale → **BASI POLINOMIALI DI BERNSTEIN/BLENDING FUNCTIONS.**

DE CASTELJAU: alg generalizzato ricorsivo curve Bézier basato su corner cutting successivi.

Curve Bézier si possono dividere e applicare test linearità, poi si possono unire per formare **CURVE DI BÉZIER A TRATTI** con vincoli **CONTINUITÀ C0** (posizionale), **C1** (deriv), **G1** (=tang, =verso tang, != mod tang), **C2** (deriv'') → alcuni CP diventano fissi.

SUPERFICIE PARAMETRICA = spazio param 2D x superfici a topologia rettangolare (Mesh poli nVert=nParam, sup param ↓ param)

SUPERFICIE PRODOTTO TENSORIALE: $u, v \in [0,1]$ formano griglia di CP.

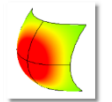
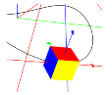
PATCH BEZIER = estensione curva Bézier. Se $n=m=3$, **CUBIC BEZIER PATCH** (16 CP): $\forall$ riga 4CP, se valuto curva ottengo punti del patch →

ALG. VALUTAZIONE EFFICIENTE. PROPRIETÀ INTERPOLAZIONE: piano tang patch è ortog normale alla sup. in quel pto.

NORMALE ALLA SUPERFICIE = prod vett normalizzato della derivata parziale in u e v.

TASSELLAZIONE (discretizzazz sup in griglia) → UNIFORME (valutazione esatta), ADATTIVA, MISTA (adatt + unif)

CONS = curve 2D di Bézier su un patch, hanno 2 curvature principali: GAUSSIANA = $k_1 k_2$, MEDIA $(k_1 + k_2)/2$ → classificazione



Costruire curve con continuità assegnata nello **SPAZIO delle FUNZIONI SPLINE** con k+1 tratti, grado n polinomi, n-1 ordine continuità $\forall$ nodo e $\dim = n+k+1$, le cui basi sono **BASI DELLE FUNZIONI B-SPLINE** (gen. Bernstein).

FUNZIONI B-SPLINE = data PARTIZIONE NODALE dei nodi, costruisco **PARTIZIONE NODALE ESTESA/DELTA***.

CURVE SPLINE: f vett composte da f scalari $\in$ spazio Spline, dipendono da GRADO, PARTIZIONE NODALE, CP.

Hanno controllo locale, interne a GUSCIO CONVESSO dei CP, **GUSCIO CONVESSO LOCALE** ($\forall$ tratto), interpola estremi, invariante trasf affini. Possono essere chiuse e periodiche. Approssimano coniche.

ALG DI DE BOOR x valutazione Curve Spline: assegno t e det intervallo a cui $\in$. Divido curva ($\sim$ De Casteljau). Se uso al da nodo x_1 a nodo x_k , ottengo CURVA DI BÉZIER A TRATTI.

ALG DI BOHM (KNOT-INSERTION): alg corner cutting utile per editare curva, convertire Spline → Bézier, **RAFFINAMENTO** (nuovo nodo nel pto medio $\forall$ tratto)



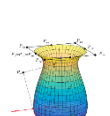
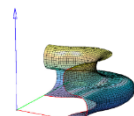
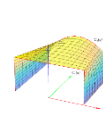
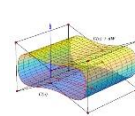
SUPERFICIE SPLINE: prod tensoriale 2 SPAZI SPLINE, crea griglia controllo ottenuta con **raffinamenti successivi**

(approssimazione) o con **tassellazione** (valutazione esatta). **QUADRATINO = BÉZIER PATCH**, controllo locale. **SUP TRIMMATE** (composizione solidi). Approssimano quadriche.

CURVE NURBS (Non Uniform Rational B-Splines): curve spline 4D proiettate nello spazio 3D. Rappresentano esattamente coniche.

SUPERFICI NURBS: sup spline 4D proiettate nello spazio 3D. Rappresentano esattamente quadriche. Qualità ↑, costo ↓.

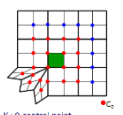
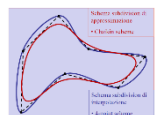
- **SUPERFICI DI ESTRUSIONE:** curva traslata
- **SUPERFICI RIGATE:** 2 curve nurbs in cui unisco pti con =t
- **SUPERFICI DI SWINGING:** curva traiettoria + curva spline nurbs
- **SUPERFICI DI ROTAZIONE:** swinging ma traiettoria è circolare (vaso)



SUDDIVISIONE: proc iterativo dove mesh raffinata fino a curva smooth. Vantaggi spline, no buchi/strappi, spesso no formule.

SCHEMI SUDDIVISIONE usano **MASCHERE** rappresentate da diagrammi.

- Triangolare/quadrilatero
- **APPROSSIMAZIONE/INTERPOLAZIONE**
 - App → **CHAIKIN SCHEME:** vert orig NON su sup, taglio a frazione fissa = ottengo spline quadratica
 - Interp → **4POINT SCHEME:** vert orig su sup, uso indici dispari per interpolare = curva non matematica
- **STAZIONARIO** ($\forall$ livello=regole) /NON STAZIONARIO
- **UNIFORME** ($\forall$ regione=regole) /NON UNIFORME



ALGORITMI DI STAM: valutazione esatta patch regolari e irregolari

SUPERFICI DI SUDDIVISIONE DI CATMULL-CLARK (GERI'S GAME) → mesh = sup spline bicubica, **mesh raffinata** = costruz incrementale $\sim$ knot insertion. 1) face points, 2) edge points, 3) vertex point, 4) formo nuovi edges, 5) ricomincio.